E-Commerce Web Application

Complete Solution for Task 3

A basic online store with product management, cart, checkout, user login, role-based access, backend APIs, database integration and order tracking.

Requirement	Implemented
Product catalog	Yes
Cart + checkout	Yes
Login + Admin/User roles	Yes
Product & order APIs	Yes
SQLite database	Yes
Order tracking	Yes
Responsive design	Yes

How to run

1. Extract the ZIP and open it in VS Code.
2. Open Terminal.
3. Run **npm install**.
4. Run **npm start**.
5. Open **http://localhost:3000**.
6. Register, login, add products to cart and checkout.

Project structure

```
ecommerce-web-application/
■■■ package.json
■■■ server.js
■■■ README.md
■■■ public/
    ■■■ index.html
    ■■■ style.css
    ■■■ script.js
```

package.json

```
{
  "name": "ecommerce-web-application",
  "version": "1.0.0",
  "scripts": { "start": "node server.js" },
  "dependencies": {
    "bcryptjs": "^2.4.3",
    "better-sqlite3": "^11.3.0",
    "cors": "^2.8.5",
    "express": "^4.21.0",
    "jsonwebtoken": "^9.0.2"
  }
}
```

server.js

```
const express = require("express");
const cors = require("cors");
const path = require("path");
const bcrypt = require("bcryptjs");
const jwt = require("jsonwebtoken");
const Database = require("better-sqlite3");

const app = express();
const PORT = 3000;
const SECRET = "change_this_secret_in_production";
const db = new Database("shop.db");
db.pragma("foreign_keys = ON");

db.exec(`
CREATE TABLE IF NOT EXISTS users(
 id INTEGER PRIMARY KEY AUTOINCREMENT,
 name TEXT NOT NULL,
 email TEXT UNIQUE NOT NULL,
 password TEXT NOT NULL,
 role TEXT NOT NULL DEFAULT 'user'
);
CREATE TABLE IF NOT EXISTS products(
 id INTEGER PRIMARY KEY AUTOINCREMENT,
 name TEXT NOT NULL,
 description TEXT DEFAULT '',
 price REAL NOT NULL,
 image TEXT DEFAULT '',
 stock INTEGER NOT NULL DEFAULT 0
);
CREATE TABLE IF NOT EXISTS orders(
 id INTEGER PRIMARY KEY AUTOINCREMENT,
 user_id INTEGER NOT NULL,
 total REAL NOT NULL,
 status TEXT NOT NULL DEFAULT 'Placed',
 created_at TEXT DEFAULT CURRENT_TIMESTAMP,
 FOREIGN KEY(user_id) REFERENCES users(id)
);
CREATE TABLE IF NOT EXISTS order_items(
 id INTEGER PRIMARY KEY AUTOINCREMENT,
 order_id INTEGER NOT NULL,
 product_id INTEGER NOT NULL,
 quantity INTEGER NOT NULL,
 price REAL NOT NULL,
 FOREIGN KEY(order_id) REFERENCES orders(id) ON DELETE CASCADE,
 FOREIGN KEY(product_id) REFERENCES products(id)
);
`);

const count = db.prepare("SELECT COUNT(*) AS c FROM products").get().c;
if (!count) {
  const add = db.prepare("INSERT INTO products(name,description,price,stock) VALUES(?,?,?,?)");
  const seed = db.transaction(() => {
    add.run("Wireless Headphones","Bluetooth over-ear headphones",1499,20);
    add.run("Smart Watch","Fitness and notification smartwatch",1999,15);
    add.run("Laptop Backpack","Water-resistant laptop backpack",899,30);
    add.run("USB-C Hub","Multi-port USB-C adapter",799,25);
  });
  seed();
}

app.use(cors());
app.use(express.json());
app.use(express.static(path.join(__dirname, "public")));

function auth(req,res,next){
  const h=req.headers.authorization;
  if(!h) return res.status(401).json({message:"Login required"});
  try { req.user=jwt.verify(h.split(" ")[1],SECRET); next(); }
  catch { res.status(401).json({message:"Invalid or expired token"}); }
}

function admin(req,res,next){
  if(req.user.role!=="admin") return res.status(403).json({message:"Admin access required"});
  next();
}

app.post("/api/register",async(req,res)=>{
  const {name,email,password}=req.body;
  if(!name||!email||!password) return res.status(400).json({message:"All fields required"});
  try{
    const hash=await bcrypt.hash(password,10);
    const r=db.prepare("INSERT INTO users(name,email,password,role) VALUES(?,?,?,'user')")
```

```
      .run(name.trim(),email.trim().toLowerCase(),hash);
    res.status(201).json({message:"Registration successful",id:r.lastInsertRowid});
  }catch(e){ res.status(409).json({message:"Email already registered"}); }
});

app.post("/api/login",async(req,res)=>{
  const {email,password}=req.body;
  const u=db.prepare("SELECT * FROM users WHERE email=?").get(email.trim().toLowerCase());
  if(!u || !(await bcrypt.compare(password,u.password)))
    return res.status(401).json({message:"Invalid email or password"});
  const token=jwt.sign({id:u.id,name:u.name,email:u.email,role:u.role},SECRET,{expiresIn:"2h"});
  res.json({token,user:{id:u.id,name:u.name,email:u.email,role:u.role}});
});

app.get("/api/products",(req,res)=>{
  res.json(db.prepare("SELECT * FROM products ORDER BY id DESC").all());
});

app.post("/api/products",auth,admin,(req,res)=>{
  const {name,description="",price,stock=0}=req.body;
  if(!name||price===undefined) return res.status(400).json({message:"Name and price required"});
  const r=db.prepare("INSERT INTO products(name,description,price,stock) VALUES(?,?,?,?)")
    .run(name,description,Number(price),Number(stock));
  res.status(201).json({message:"Product added",id:r.lastInsertRowid});
});

app.put("/api/products/:id",auth,admin,(req,res)=>{
  const {name,description="",price,stock}=req.body;
  const r=db.prepare("UPDATE products SET name=?,description=?,price=?,stock=? WHERE id=?")
    .run(name,description,Number(price),Number(stock),req.params.id);
  if(!r.changes) return res.status(404).json({message:"Product not found"});
  res.json({message:"Product updated"});
});

app.delete("/api/products/:id",auth,admin,(req,res)=>{
  const r=db.prepare("DELETE FROM products WHERE id=?").run(req.params.id);
  if(!r.changes) return res.status(404).json({message:"Product not found"});
  res.json({message:"Product deleted"});
});

app.post("/api/orders",auth,(req,res)=>{
  const {items}=req.body;
  if(!Array.isArray(items)||!items.length) return res.status(400).json({message:"Cart is empty"});

  const transaction=db.transaction(()=>{
    let total=0, rows=[];
    for(const item of items){
      const p=db.prepare("SELECT * FROM products WHERE id=?").get(item.product_id);
      const q=Number(item.quantity);
      if(!p || q<1 || q>p.stock) throw new Error("Invalid product or insufficient stock");
      total += p.price*q;
      rows.push({p,q});
    }
    const order=db.prepare("INSERT INTO orders(user_id,total,status) VALUES(?,?,?)")
      .run(req.user.id,total,"Placed");
    const addItem=db.prepare("INSERT INTO order_items(order_id,product_id,quantity,price) VALUES(?,?,?,?)");
    const reduce=db.prepare("UPDATE products SET stock=stock-? WHERE id=?");
    for(const x of rows){ addItem.run(order.lastInsertRowid,x.p.id,x.q,x.p.price); reduce.run(x.q,x.p.id); }
    return order.lastInsertRowid;
  });

  try {
    const id=transaction();
    res.status(201).json({message:"Order placed",orderId:id});
  } catch(e) { res.status(400).json({message:e.message}); }
});

app.get("/api/orders",auth,(req,res)=>{
  const orders=db.prepare("SELECT * FROM orders WHERE user_id=? ORDER BY id DESC").all(req.user.id);
  res.json(orders);
});

app.get("/api/admin/orders",auth,admin,(req,res)=>{
  res.json(db.prepare(`
    SELECT orders.id, users.name, users.email, orders.total, orders.status, orders.created_at
    FROM orders JOIN users ON users.id=orders.user_id ORDER BY orders.id DESC
  `).all());
});

app.put("/api/admin/orders/:id",auth,admin,(req,res)=>{
  const allowed=["Placed","Processing","Shipped","Delivered","Cancelled"];
  if(!allowed.includes(req.body.status)) return res.status(400).json({message:"Invalid status"});
  const r=db.prepare("UPDATE orders SET status=? WHERE id=?").run(req.body.status,req.params.id);
  if(!r.changes) return res.status(404).json({message:"Order not found"});
  res.json({message:"Order status updated"});
```

```
});

app.get("*",(req,res)=>res.sendFile(path.join(__dirname,"public","index.html")));
app.listen(PORT,()=>console.log(`E-commerce app: http://localhost:${PORT}`));
```

public/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width,initial-scale=1">
<title>MiniShop E-Commerce</title>
<link rel="stylesheet" href="style.css">
</head>
<body>
<header>
  <h1>MiniShop</h1>
  <nav>
    <button onclick="showShop()">Shop</button>
    <button onclick="showOrders()">My Orders</button>
    <button id="authBtn" onclick="openAuth()">Login</button>
    <button onclick="logout()">Logout</button>
    <span>■ <b id="cartCount">0</b></span>
  </nav>
</header>

<main>
<section id="shop">
  <h2>Product Catalog</h2>
  <div id="products" class="grid"></div>
</section>

<section id="cart" class="panel hidden">
  <h2>Your Cart</h2>
  <div id="cartItems"></div>
  <h3 id="cartTotal"></h3>
  <button onclick="checkout()">Checkout</button>
</section>

<section id="orders" class="panel hidden">
  <h2>Order Tracking</h2>
  <div id="orderList"></div>
</section>

<section id="auth" class="panel hidden">
  <h2 id="authTitle">Login</h2>
  <input id="name" placeholder="Name">
  <input id="email" type="email" placeholder="Email">
  <input id="password" type="password" placeholder="Password">
  <button onclick="submitAuth()">Login</button>
  <button class="link" onclick="toggleAuth()">Create account</button>
  <p id="msg"></p>
</section>
</main>
<script src="script.js"></script>
</body>
</html>
```

public/style.css

```
*{box-sizing:border-box}body{margin:0;font-family:Arial,sans-serif;background:#f4f7fb;color:#172033}
header{background:#2563eb;color:white;padding:16px 5%;display:flex;justify-content:space-between;align-items
nav{display:flex;align-items:center;gap:8px;flex-wrap:wrap}button{border:0;border-radius:8px;padding:10px 14
header button{background:white;color:#2563eb}main{width:min(1100px,94%);margin:25px auto}.grid{display:grid;
.product,.panel{background:white;border-radius:14px;padding:18px;box-shadow:0 4px 15px rgba(0,0,0,.08)}
.product h3{margin-top:0}.price{font-size:20px;font-weight:bold}.stock{color:#667085}.product button{width:1
.panel{max-width:700px;margin:20px auto}input{width:100%;padding:12px;margin:7px 0;border:1px solid #ccd2dc;
.hidden{display:none}.cartrow,.order{border-bottom:1px solid #e5e7eb;padding:12px 0}.link{background:transpa
.danger{background:#dc2626}@media(max-width:600px){header{padding:14px 3%}header h1{margin:0}nav{width:100%}
```

public/script.js

```
let products=[], cart=JSON.parse(localStorage.getItem("cart")||"[]"), register=false;

async function loadProducts(){
  const r=await fetch("/api/products"); products=await r.json(); renderProducts();
}
function renderProducts(){
  document.getElementById("products").innerHTML=products.map(p=>`
    <article class="product">
      <h3>${esc(p.name)}</h3><p>${esc(p.description)}</p>
      <div class="price">■${Number(p.price).toFixed(2)}</div>
      <div class="stock">Stock: ${p.stock}</div>
      <button ${p.stock<1?"disabled":""} onclick="addToCart(${p.id})">Add to Cart</button>
    </article>`).join("");
  updateCart();
}
function addToCart(id){
  const item=cart.find(x=>x.product_id===id);
  if(item)item.quantity++; else cart.push({product_id:id,quantity:1});
  saveCart(); alert("Added to cart");
}
function saveCart(){localStorage.setItem("cart",JSON.stringify(cart));updateCart();}
function updateCart(){
  document.getElementById("cartCount").textContent=cart.reduce((a,x)=>a+x.quantity,0);
  const box=document.getElementById("cartItems");
  let total=0;
  box.innerHTML=cart.map(x=>{
    const p=products.find(y=>y.id===x.product_id); if(!p)return "";
    total+=p.price*x.quantity;
    return `<div class="cartrow"><b>${esc(p.name)}</b> × ${x.quantity} — ■${(p.price*x.quantity).toFixed(2)}
      <button class="danger" onclick="removeItem(${p.id})">Remove</button></div>`;
  }).join("")||"<p>Cart is empty.</p>";
  document.getElementById("cartTotal").textContent="Total: ■"+total.toFixed(2);
}
function removeItem(id){cart=cart.filter(x=>x.product_id!==id);saveCart();}
function showShop(){hideAll();document.getElementById("shop").classList.remove("hidden");document.getElement
function showOrders(){hideAll();document.getElementById("orders").classList.remove("hidden");loadOrders();}
function hideAll(){["shop","cart","orders","auth"].forEach(id=>document.getElementById(id).classList.add("hi
function openAuth(){hideAll();document.getElementById("auth").classList.remove("hidden");}
function toggleAuth(){register=!register;document.getElementById("authTitle").textContent=register?"Create A
async function submitAuth(){
  const body={email:email.value.trim(),password:password.value};
  if(register)body.name=name.value.trim();
  const r=await fetch(register?"/api/register":"/api/login",{method:"POST",headers:{"Content-Type":"applicat
  const d=await r.json();msg.textContent=d.message;
  if(r.ok&&!register){localStorage.setItem("token",d.token);showShop();}
  if(r.ok&&register){toggleAuth();}
}
async function checkout(){
  const token=localStorage.getItem("token"); if(!token){openAuth();return;}
  if(!cart.length)return alert("Cart is empty");
  const r=await fetch("/api/orders",{method:"POST",headers:{"Content-Type":"application/json","Authorization
  const d=await r.json(); if(!r.ok)return alert(d.message);
  cart=[];saveCart();await loadProducts();alert("Order placed successfully. Order ID: "+d.orderId);showOrder
}
async function loadOrders(){
  const token=localStorage.getItem("token"); if(!token){openAuth();return;}
  const r=await fetch("/api/orders",{headers:{Authorization:"Bearer "+token}}); if(!r.ok)return openAuth();
  const data=await r.json();
  orderList.innerHTML=data.length?data.map(o=>`<div class="order"><b>Order #${o.id}</b><br>Total: ■${o.total
}
function logout(){localStorage.removeItem("token");alert("Logged out");showShop();}
function esc(s){return String(s).replace(/[&<>"']/g,c=>({"&":"&amp;","<":"&lt;",">":"&gt;",'"':"&quot;","'":
loadProducts();
```

README.md

```
# E-Commerce Web Application

## Requirements implemented
- Product catalog
- Add to cart
- Checkout/order creation
- User registration and login
- Role-based access in backend (Admin/User)
- REST APIs for products and orders
- SQLite database integration
- Order tracking
- Responsive web/mobile design

## Run
Install Node.js LTS, open this folder in VS Code, then run:

npm install
npm start

Open http://localhost:3000

## API
POST /api/register
POST /api/login
GET /api/products
POST /api/products (Admin)
PUT /api/products/:id (Admin)
DELETE /api/products/:id (Admin)
POST /api/orders
GET /api/orders
GET /api/admin/orders (Admin)
PUT /api/admin/orders/:id (Admin)

## Admin
The backend supports an Admin role. For demonstration, a user can be changed to admin directly in SQLite:
UPDATE users SET role='admin' WHERE email='your-email@example.com';

For production, use environment variables for JWT secret, HTTPS, validation and a production database.
```

Submission checklist

✓ Product catalog
✓ Add to cart and checkout
✓ User login
✓ Role-based Admin/User authorization in backend
✓ Product and order REST APIs
✓ Database integration
✓ Order tracking
✓ Responsive design
✓ README setup instructions